

```
In [2]: import os
import tensorflow as tf
from random import seed, random
from shutil import copyfile
from os import makedirs, listdir
from matplotlib import pyplot as plt
from matplotlib.image import imread

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import SGD, Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
In [3]: # Print the current working directory
print("Current working directory: {}".format(os.getcwd()))

# Change the current working directory
os.chdir('C:/Users/massi/OneDrive/Desktop/Dogs-Cats')

# Print the current working directory
print("Current working directory: {}".format(os.getcwd()))
```

Current working directory: C:\Users\massi
Current working directory: C:\Users\massi\OneDrive\Desktop\Dogs-Cats

Visualizziamo le prime 9 immagini dei cani presenti nella cartella dogs

```
In [4]: folder = 'C:/Users/massi/OneDrive/Desktop/Dogs-Cats/training_set/dogs/'

plt.figure(figsize=(10, 10))

# Visualizza le immagini dog.1.jpg fino a dog.9.jpg
for i in range(1, 10): # attenzione: parte da 1, non da 0
    filename = os.path.join(folder, f'dog.{i}.jpg')
    image = imread(filename)
    plt.subplot(3, 3, i)
    plt.imshow(image)
    plt.title(f'dog.{i}.jpg')
    plt.axis('off')

plt.tight_layout()
plt.show()
```

dog.1.jpg



dog.2.jpg



dog.3.jpg



dog.4.jpg



dog.5.jpg



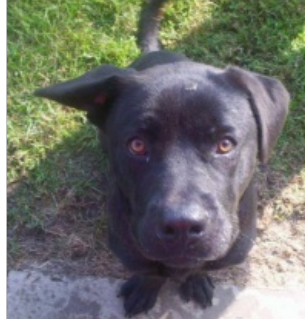
dog.6.jpg



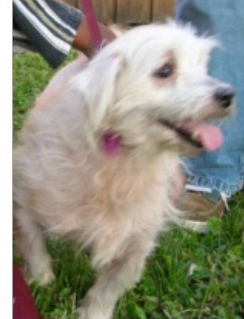
dog.7.jpg



dog.8.jpg



dog.9.jpg



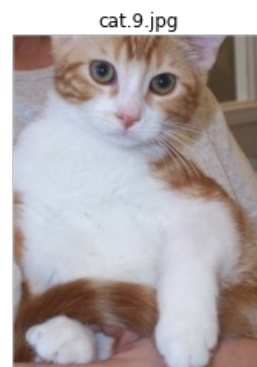
Visualizziamo le prime 9 immagini dei gatti presenti nella cartella cats

```
In [5]: folder = 'C:/Users/massi/OneDrive/Desktop/Dogs-Cats/training_set/cats/'

plt.figure(figsize=(10, 10))

# Visualizza le immagini dog.1.jpg fino a dog.9.jpg
for i in range(1, 10): # attenzione: parte da 1, non da 0
    filename = os.path.join(folder, f'cat.{i}.jpg')
    image = imread(filename)
    plt.subplot(3, 3, i)
    plt.imshow(image)
    plt.title(f'cat.{i}.jpg')
    plt.axis('off')

plt.tight_layout()
plt.show()
```



Definiamo una rete neurale in cui la dimensione delle immagini è 200x200 e 3 sono i canali, rispettivamente, R-G-B.

Il filtro è una matrice 3x3 di pesi (per ogni canale input, quindi 3x3x3 in totale per immagini RGB).

Questo filtro viene fatto "scorrere" (slide) sull'immagine, pixel per pixel, applicando un'operazione di prodotto scalare tra i pesi del filtro e i pixel corrispondenti dell'immagine.

Il risultato di questa operazione per ogni posizione è un singolo valore che compone la mappa di attivazione (feature map) prodotta da quel filtro.

Ogni filtro scorre sull'intera immagine, ma con pesi diversi, producendo 8 mappe di attivazione differenti, ognuna evidenziando caratteristiche diverse.

Dopo che ogni filtro convoluzionale ha calcolato la sua mappa di attivazione (valori reali, positivi o negativi), la ReLU trasforma ogni valore negativo in 0 e lascia invariati i positivi.

Quindi la feature map risultante è "pulita", più facile da interpretare e più efficiente da elaborare per i layer successivi.

MaxPooling2D() -> È un layer di pooling usato nelle CNN per ridurre la dimensione spaziale (altezza e larghezza) delle feature map, mantenendo però le informazioni più importanti.

Divide la feature map in piccole finestre (tipicamente 2x2) e per ogni finestra, prende il valore massimo (da qui il nome max pooling)

Successivamente abbiamo il secondo layer convolutivo in cui abbiamo 4 filtri (8 nel primo layer convolutivo), ognuno è una matrice 3x3 di pesi

Come in precedenza, anche in questo caso successivamente avremo il layer di pooling.

Flatten() -> è un layer che "appiattisce" una struttura dati multidimensionale (come una matrice 2D o 3D) in un vettore 1D.

Nelle reti convoluzionali, dopo aver estratto le feature tramite i layer convoluzionali e di pooling, otteniamo feature map multidimensionali.

Prima di passare ai layer fully connected (Dense), dobbiamo trasformare questa struttura in un vettore monodimensionale, e qui entra in gioco Flatten().

Dropout(0.5) -> è una tecnica di regolarizzazione usata nelle reti neurali per evitare l'overfitting.

Durante l'allenamento, ad ogni passaggio (batch), il layer "spegne" casualmente il 50% dei neuroni del layer precedente (se dropout=0.5).

I neuroni "spenti" non contribuiscono né al forward pass né al backpropagation in quel passaggio.

Questa "disattivazione casuale" costringe la rete a non dipendere troppo da singoli neuroni, rendendo il modello più robusto.

Dense(1) -> è un layer completamente connesso (fully connected) con 1 neurone.

activation='sigmoid' -> applica la funzione di attivazione sigmoide.

Quando si fa una classificazione binaria (esempio: gatto vs cane), il neurone produce un output tra 0 e 1, che rappresenta la probabilità che l'input appartenga alla classe positiva (ad esempio "gatto").

loss='binary_crossentropy' -> Funzione di perdita adatta per problemi di classificazione binaria. Misura la differenza tra la probabilità predetta e quella vera (etichetta).

optimizer=SGD(learning_rate=0.01) -> Utilizza lo Stochastic Gradient Descent come algoritmo di ottimizzazione con un tasso di apprendimento (learning rate) pari a 0.01.

metrics=['accuracy'] -> Durante l'addestramento e la valutazione, calcola e mostra la precisione (accuracy), cioè la percentuale di predizioni corrette.

```
In [6]: # Funzione per definire il modello CNN di base
def define_model():
    model = Sequential([
        Conv2D(8, (3, 3), padding='same', activation='relu', input_shape=(200, 200, 3)),
        MaxPooling2D(),
        Conv2D(4, (3, 3), padding='same', activation='relu'),
        MaxPooling2D(),
        Flatten(),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])

    # Compilo il modello con loss per classificazione binaria, optimizer SGD con learning rate 0.01, e accurate
    model.compile(loss='binary_crossentropy', optimizer=SGD(learning_rate=0.01), metrics=['accuracy'])
    return model

model = define_model()
model.summary()
```

```
C:\Users\massi\anaconda3\lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 200, 200, 8)	224
max_pooling2d (MaxPooling2D)	(None, 100, 100, 8)	0
conv2d_1 (Conv2D)	(None, 100, 100, 4)	292
max_pooling2d_1 (MaxPooling2D)	(None, 50, 50, 4)	0
flatten (Flatten)	(None, 10000)	0
dropout (Dropout)	(None, 10000)	0
dense (Dense)	(None, 1)	10,001

Total params: 10,517 (41.08 KB)

Trainable params: 10,517 (41.08 KB)

Non-trainable params: 0 (0.00 B)

```
In [7]: # Funzione per visualizzare le curve di apprendimento dopo il training del modello
def summarize_diagnostics(history):
    # Prima figura: perdita (loss) su training e validation
    plt.subplot(211)
    plt.title('Cross Entropy Loss')
    plt.plot(history.history['loss'], color='blue', label='train') # perdita training
    plt.plot(history.history['val_loss'], color='orange', label='validation') # perdita validation
    plt.legend()

    # Seconda figura: accuratezza su training e validation
```

```

plt.subplot(212)
plt.title('Classification Accuracy')
plt.plot(history.history['accuracy'], color='blue', label='train') # accuracy training
plt.plot(history.history['val_accuracy'], color='orange', label='validation') # accuracy validation
plt.legend()

# Mostra i grafici
plt.show()

```

batch_size=5 -> Indica quante immagini (o campioni) vengono processate insieme prima di aggiornare i pesi della rete. In questo caso, ogni volta che la rete riceve 5 immagini, calcola il gradiente e aggiorna i pesi.

epochs=5 -> Indica quante volte l'intero dataset di training viene "passato" attraverso la rete. Il modello vedrà ogni immagine del training set 5 volte durante l'allenamento. Più epoche possono migliorare la capacità di apprendimento ma possono anche causare overfitting.

```

In [11]: def run_test_harness():
    model = define_model()

    datagen = ImageDataGenerator(rescale=1.0 / 255.0)

    # Usa percorsi assoluti o verifica che esistano i percorsi relativi
    train_dir = r'C:\Users\massi\OneDrive\Desktop\ Dogs-Cats\training_set'
    test_dir = r'C:\Users\massi\OneDrive\Desktop\ Dogs-Cats\test_set'

    # Verifica che le cartelle esistano
    assert os.path.isdir(train_dir), f"Cartella non trovata: {train_dir}"
    assert os.path.isdir(test_dir), f"Cartella non trovata: {test_dir}"

    train_it = datagen.flow_from_directory(
        train_dir,
        class_mode='binary',
        batch_size=5,
        target_size=(200, 200)
    )

    test_it = datagen.flow_from_directory(
        test_dir,
        class_mode='binary',
        batch_size=5,
        target_size=(200, 200)
    )

    history = model.fit(
        train_it,
        steps_per_epoch=len(train_it),
        validation_data=test_it,
        validation_steps=len(test_it),
        epochs=5,
        verbose=1
    )

    _, acc = model.evaluate(test_it, steps=len(test_it), verbose=0)
    print(f'> Accuracy sul test set: {acc * 100:.3f}%')

    summarize_diagnostics(history)

run_test_harness()

```

Found 8005 images belonging to 2 classes.

Found 1826 images belonging to 2 classes.

C:\Users\massi\anaconda3\lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

```
self.warn_if_super_not_called()
```

Epoch 1/5

1601/1601 ————— 242s 150ms/step - accuracy: 0.5331 - loss: 0.6961 - val_accuracy: 0.6320 - val_loss: 0.6506

Epoch 2/5

1601/1601 ————— 70s 44ms/step - accuracy: 0.6029 - loss: 0.6550 - val_accuracy: 0.5843 - val_loss: 0.6643

Epoch 3/5

1601/1601 ————— 62s 39ms/step - accuracy: 0.6476 - loss: 0.6247 - val_accuracy: 0.6265 - val_loss: 0.6477

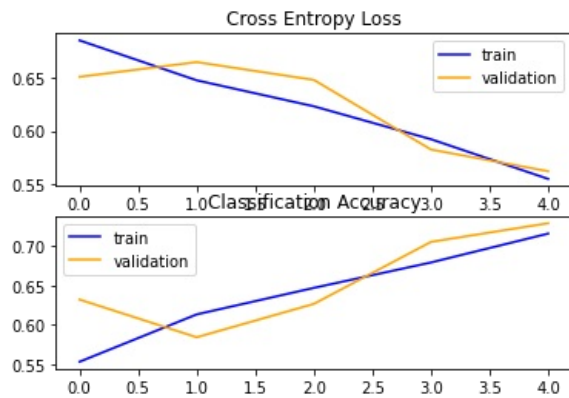
Epoch 4/5

1601/1601 ————— 64s 40ms/step - accuracy: 0.6711 - loss: 0.6038 - val_accuracy: 0.7048 - val_loss: 0.5827

Epoch 5/5

1601/1601 ————— 96s 49ms/step - accuracy: 0.7110 - loss: 0.5555 - val_accuracy: 0.7284 - val_loss: 0.5625

> Accuracy sul test set: 72.837%



Dall'analisi dei grafici si osserva che, al crescere del numero di epoche, l'accuratezza tende ad aumentare. Con 5 epoche, il modello raggiunge un'accuracy del 71% sul set di addestramento e del 72% sul set di validazione, indicando una buona capacità di generalizzazione già dopo poche epoche.

MIGLIORIAMO L'ACCURATEZZA PREVISIVA

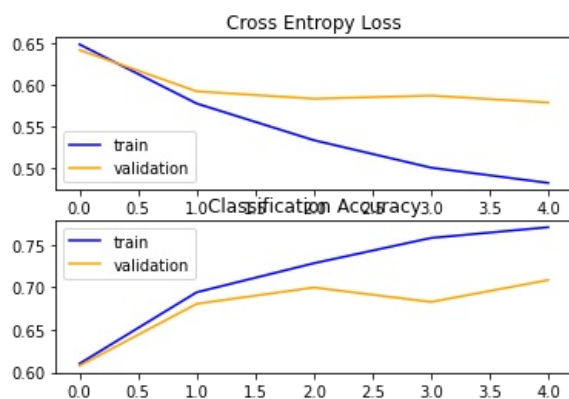
Per cercare di migliorare l'accuratezza predittiva della rete convolutiva, è possibile considerare l'impiego di algoritmi di ottimizzazione alternativi allo SGD (Stochastic Gradient Descent). Un'opzione efficace è l'ottimizzatore Adam, che combina i vantaggi del momentum e dell'adaptive learning rate, consentendo in molti casi una convergenza più rapida e stabile. L'utilizzo di Adam al posto di SGD può portare a prestazioni migliori, soprattutto in fase di training, ed è quindi una strategia da prendere in considerazione per ottimizzare il modello.

```
In [22]: def define_model():
    model = Sequential([
        Conv2D(8, (3, 3), padding='same', activation='relu', input_shape=(200, 200, 3)),
        MaxPooling2D(),
        Conv2D(4, (3, 3), padding='same', activation='relu'),
        MaxPooling2D(),
        Flatten(),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])

    # Compilo il modello con loss per classificazione binaria, optimizer Adam, e accuratezza
    model.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.001), metrics=['accuracy'])
    return model
```

```
In [23]: run_test_harness()

Found 8005 images belonging to 2 classes.
Found 1826 images belonging to 2 classes.
Epoch 1/5
1601/1601 ————— 70s 42ms/step - accuracy: 0.5723 - loss: 0.6698 - val_accuracy: 0.6079 - val_loss: 0.6422
Epoch 2/5
1601/1601 ————— 58s 36ms/step - accuracy: 0.6826 - loss: 0.5926 - val_accuracy: 0.6807 - val_loss: 0.5923
Epoch 3/5
1601/1601 ————— 74s 46ms/step - accuracy: 0.7315 - loss: 0.5326 - val_accuracy: 0.6999 - val_loss: 0.5834
Epoch 4/5
1601/1601 ————— 52s 32ms/step - accuracy: 0.7661 - loss: 0.4922 - val_accuracy: 0.6829 - val_loss: 0.5871
Epoch 5/5
1601/1601 ————— 88s 55ms/step - accuracy: 0.7672 - loss: 0.4819 - val_accuracy: 0.7087 - val_loss: 0.5788
> Accuracy sul test set: 70.865%
```



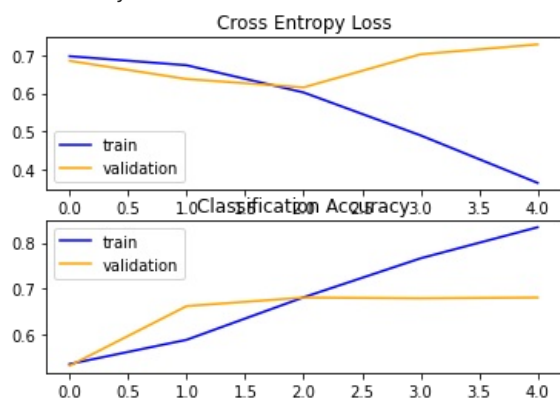
Aggiungere più layer e più filtri

```
In [24]: def define_model():
model = Sequential([
    Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(200, 200, 3)),
    MaxPooling2D(),
    Conv2D(64, (3, 3), padding='same', activation='relu'),
    MaxPooling2D(),
    Flatten(),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compilo il modello con loss per classificazione binaria, optimizer Adam con learning rate 0.01, e accuracy
model.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.001), metrics=['accuracy'])
return model
```

```
In [25]: run_test_harness()

Found 8005 images belonging to 2 classes.
Found 1826 images belonging to 2 classes.
Epoch 1/5
1601/1601 ————— 371s 231ms/step - accuracy: 0.5185 - loss: 0.7084 - val_accuracy: 0.5318 - val_loss: 0.6853
Epoch 2/5
1601/1601 ————— 378s 236ms/step - accuracy: 0.5724 - loss: 0.6829 - val_accuracy: 0.6621 - val_loss: 0.6375
Epoch 3/5
1601/1601 ————— 372s 232ms/step - accuracy: 0.6671 - loss: 0.6089 - val_accuracy: 0.6807 - val_loss: 0.6156
Epoch 4/5
1601/1601 ————— 368s 230ms/step - accuracy: 0.7650 - loss: 0.4912 - val_accuracy: 0.6791 - val_loss: 0.7026
Epoch 5/5
1601/1601 ————— 389s 243ms/step - accuracy: 0.8441 - loss: 0.3592 - val_accuracy: 0.6807 - val_loss: 0.7284
> Accuracy sul test set: 68.072%
```



L'accuratezza sul training set tende ad aumentare quando si adotta l'algoritmo di ottimizzazione Adam al posto di SGD. grazie alla sua

...accuratezza del training set tende ad aumentare quando si adottano algoritmi di ottimizzazione Adam o RMSProp, grazie alla sua capacità di adattare dinamicamente il learning rate e accelerare la convergenza. Successivamente, introducendo Adam e contemporaneamente aumentando il numero di layer nella rete, si ottiene un ulteriore incremento della capacità della rete di apprendere rappresentazioni complesse.

Tuttavia, in entrambi i casi, si osserva che l'accuratezza sul test set tende a diminuire leggermente rispetto al modello CNN di base. Questo comportamento può indicare un inizio di overfitting, in cui il modello si adatta molto bene ai dati di addestramento ma perde capacità di generalizzazione su dati nuovi. In sintesi, l'adozione di Adam e l'aumento della complessità del modello migliorano le performance sul training set, ma non necessariamente sull'insieme di validazione, suggerendo la necessità di ulteriori tecniche di regolarizzazione o data augmentation per migliorare la generalizzazione.

Data Augmentation

La data augmentation va applicata solo al training set, non al validation/test set. Questo perché il modello deve essere valutato su dati non alterati.

La data augmentation è una tecnica che crea versioni leggermente modificate delle immagini del dataset per:

- 1) Aumentare la quantità apparente di dati
- 2) Ridurre l'overfitting
- 3) Migliorare la capacità di generalizzazione del modello

rotation_range -> ruota le immagini casualmente fino a $\pm 20^\circ$

width_shift_range e height_shift_range -> trasla leggermente in larghezza e altezza

shear_range=0.1 -> Applica una distorsione sull'immagine, cioè una trasformazione che "inclina" l'immagine. Serve a rendere il modello robusto a deformazioni geometriche.

zoom_range -> applico un piccolo zoom

horizontal_flip=True -> riflette orizzontalmente alcune immagini

fill_mode='nearest' -> Quando l'immagine viene trasformata (es. ruotata o traslata), alcune aree diventano vuote. Con "nearest" questi spazi vuoti vengono riempiti copiando i pixel più vicini.

```
In [27]: def run_test_harness():
    model = define_model()

    # Data augmentation solo per il training
    train_datagen = ImageDataGenerator(
        rescale=1.0 / 255.0,
        rotation_range=20,
        width_shift_range=0.1,
        height_shift_range=0.1,
        shear_range=0.1,
        zoom_range=0.1,
        horizontal_flip=True,
        fill_mode='nearest'
    )

    # Nessuna augmentation per il test/validation set
    test_datagen = ImageDataGenerator(rescale=1.0 / 255.0)

    # Usa percorsi assoluti o verifica che esistano i percorsi relativi
    train_dir = r'C:\Users\massi\OneDrive\Desktop\Dogs-Cats\training_set'
    test_dir = r'C:\Users\massi\OneDrive\Desktop\Dogs-Cats\test_set'

    # Verifica che le cartelle esistano
    assert os.path.isdir(train_dir), f"Cartella non trovata: {train_dir}"
    assert os.path.isdir(test_dir), f"Cartella non trovata: {test_dir}"

    train_it = train_datagen.flow_from_directory(
        train_dir,
        class_mode='binary',
        batch_size=32,
        target_size=(200, 200))

    test_it = test_datagen.flow_from_directory(
        test_dir,
        class_mode='binary',
        batch_size=32,
        target_size=(200, 200))

    history = model.fit(
        train_it,
        steps_per_epoch=len(train_it),
```



```

validation_data=test_it,
validation_steps=len(test_it),
epochs=5,
verbose=1
)

_, acc = model.evaluate(test_it, steps=len(test_it), verbose=0)
print(f'> Accuracy sul test set: {acc * 100:.3f}%')

summarize_diagnostics(history)

run_test_harness()

```

Found 8005 images belonging to 2 classes.

Found 1826 images belonging to 2 classes.

Epoch 1/5

251/251 ————— 230s 911ms/step - accuracy: 0.5068 - loss: 0.9170 - val_accuracy: 0.6216 - val_loss: 0.6422

Epoch 2/5

251/251 ————— 222s 882ms/step - accuracy: 0.6094 - loss: 0.6487 - val_accuracy: 0.6046 - val_loss: 0.6719

Epoch 3/5

251/251 ————— 226s 899ms/step - accuracy: 0.6324 - loss: 0.6302 - val_accuracy: 0.6769 - val_loss: 0.6113

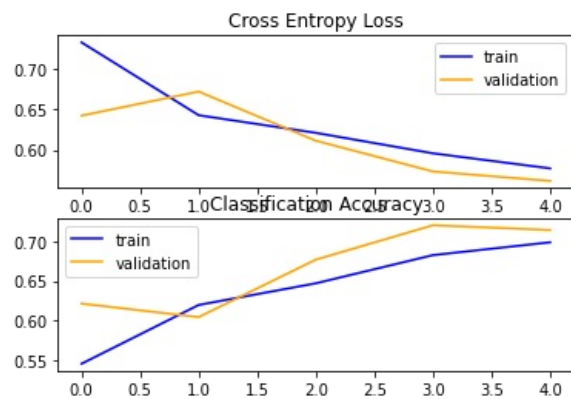
Epoch 4/5

251/251 ————— 217s 863ms/step - accuracy: 0.6821 - loss: 0.5967 - val_accuracy: 0.7202 - val_loss: 0.5734

Epoch 5/5

251/251 ————— 225s 896ms/step - accuracy: 0.7012 - loss: 0.5756 - val_accuracy: 0.7141 - val_loss: 0.5617

> Accuracy sul test set: 71.413%



L'aggiunta della data augmentation ha introdotto una maggiore varietà nei dati di addestramento, simulando nuove immagini a partire da quelle originali. Questo ha portato a una leggera diminuzione dell'accuratezza sul training set, poiché il modello si trova a dover apprendere da dati più complessi e vari. Tuttavia, proprio grazie a questa maggiore variabilità, il modello ha appreso in modo più robusto, evitando di sovradattarsi ai dati di training.

Il risultato è un'accuratezza superiore sul validation/test set, a conferma di una migliore capacità di generalizzazione del modello. In sintesi, la data augmentation ha ridotto l'overfitting e migliorato le performance previsive su dati non visti.

Batch Normalization

```

In [35]: # Funzione per definire un modello CNN con Batch Normalization
def define_model():
    model = Sequential([
        Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(200, 200, 3)),
        BatchNormalization(),
        MaxPooling2D(),

        Conv2D(64, (3, 3), padding='same', activation='relu'),
        BatchNormalization(),
        MaxPooling2D(),

        Conv2D(128, (3, 3), padding='same', activation='relu'),
        BatchNormalization(),
        MaxPooling2D(),

        Flatten(),
        Dropout(0.5),
        Dense(128, activation='relu'),
        BatchNormalization(),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])

    # Compilazione del modello con Adam
    model.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.0001), metrics=['accuracy'])
    return model

```

```
In [30]: run_test_harness()
```

Found 8005 images belonging to 2 classes.

Found 1826 images belonging to 2 classes.

Epoch 1/5

251/251 ————— 528s 2s/step - accuracy: 0.5985 - loss: 0.9241 - val_accuracy: 0.5537 - val_loss: 1.3305

Epoch 2/5

251/251 ————— 528s 2s/step - accuracy: 0.6564 - loss: 0.7229 - val_accuracy: 0.6791 - val_loss: 0.6462

Epoch 3/5

251/251 ————— 534s 2s/step - accuracy: 0.6930 - loss: 0.6605 - val_accuracy: 0.7552 - val_loss: 0.5387

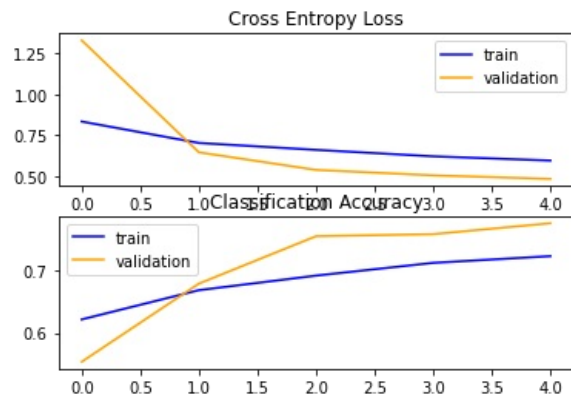
Epoch 4/5

251/251 ————— 543s 2s/step - accuracy: 0.7101 - loss: 0.6289 - val_accuracy: 0.7585 - val_loss: 0.5054

Epoch 5/5

251/251 ————— 579s 2s/step - accuracy: 0.7229 - loss: 0.5944 - val_accuracy: 0.7760 - val_loss: 0.4837

> Accuracy sul test set: 77.601%



L'introduzione della Batch Normalization dopo ogni strato convoluzionale migliora la stabilità dell'addestramento, riduce la covariate shift interna e rende il modello più robusto. In combinazione con la data augmentation, l'ottimizzatore Adam e una rete più profonda, ciò consente una migliore generalizzazione, anche se l'accuratezza sul training set può risultare leggermente più bassa. Tuttavia, il guadagno in accuratezza sul validation/test set evidenzia una rete meno soggetta a overfitting e con performance previsive superiori su dati non visti.

Previsioni con il modello migliorato

Dopo aver costruito e addestrato un modello CNN migliorato, che include:

- 1) Più filtri nelle convoluzioni (rispetto al modello base)
- 2) Batch Normalization, per stabilizzare e accelerare l'apprendimento riducendo l'effetto del covariate shift interno
- 3) L'ottimizzatore Adam, più efficace rispetto a SGD nel gestire learning rate adattivi

Si procede a utilizzare questo modello per effettuare previsioni su nuove immagini, ad esempio una foto di un cane e una di un gatto.

```
In [44]: def run_test_harness():
model = define_model()

# Data augmentation solo per il training
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255.0,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Nessuna augmentation per il test/validation set
test_datagen = ImageDataGenerator(rescale=1.0 / 255.0)

# Percorsi
train_dir = r'C:\Users\massi\OneDrive\Desktop\Dogs-Cats\training_set'
test_dir = r'C:\Users\massi\OneDrive\Desktop\Dogs-Cats\test_set'

# Verifica cartelle
assert os.path.isdir(train_dir), f"Cartella non trovata: {train_dir}"
assert os.path.isdir(test_dir), f"Cartella non trovata: {test_dir}"
```

```

# Generatori
train_it = train_datagen.flow_from_directory(
    train_dir,
    class_mode='binary',
    batch_size=32,
    target_size=(200, 200)
)

test_it = test_datagen.flow_from_directory(
    test_dir,
    class_mode='binary',
    batch_size=32,
    target_size=(200, 200)
)

# Addestramento
model.fit(
    train_it,
    steps_per_epoch=len(train_it),
    validation_data=test_it,
    validation_steps=len(test_it),
    epochs=5,
    verbose=1
)

return model # restituisce il modello allenato

```

In [45]: final_model = run_test_harness()

```

Found 8005 images belonging to 2 classes.
Found 1826 images belonging to 2 classes.
Epoch 1/5
251/251 ————— 626s 2s/step - accuracy: 0.5992 - loss: 0.9070 - val_accuracy: 0.5537 - val_loss: 1.5627
Epoch 2/5
251/251 ————— 516s 2s/step - accuracy: 0.6726 - loss: 0.7106 - val_accuracy: 0.6895 - val_loss: 0.6443
Epoch 3/5
251/251 ————— 523s 2s/step - accuracy: 0.7176 - loss: 0.6203 - val_accuracy: 0.7694 - val_loss: 0.4874
Epoch 4/5
251/251 ————— 526s 2s/step - accuracy: 0.7065 - loss: 0.6353 - val_accuracy: 0.7766 - val_loss: 0.4761
Epoch 5/5
251/251 ————— 527s 2s/step - accuracy: 0.7217 - loss: 0.5841 - val_accuracy: 0.7870 - val_loss: 0.4638

```

In [46]:

```

# Funzione per caricare e preprocessare l'immagine
def load_image(file_name):
    raw = tf.io.read_file(file_name)
    # Decodifica l'immagine (usa decode_png se il file è .png)
    img = tf.image.decode_jpeg(raw, channels=3)
    # Ridimensiona a 200x200 (match con input del modello)
    img = tf.image.resize(img, [200, 200])
    # Normalizza i pixel in [0, 1]
    img = img / 255.0
    return img

# Percorso immagine da testare
img_path = 'C:/Users/massi/OneDrive/Desktop/Dogs-Cats/Prove/cat.jpg'

# Carica immagine
my_image = load_image(img_path)

# Mostra immagine
plt.imshow(my_image)
plt.axis('off')
plt.show()

# Espandi dimensione per batch (shape diventa [1, 200, 200, 3])
my_batch = tf.expand_dims(my_image, 0)

# Fai la previsione (restituisce probabilità)
probs = final_model.predict(my_batch)

# Interpreta il risultato
if probs[0][0] > 0.5:
    print(f"L'immagine è classificata come **dog** ({probs[0][0]:.2f})")
else:
    print(f"L'immagine è classificata come **cat** ({1 - probs[0][0]:.2f})")

# Carica immagine
img = load_image('C:/Users/massi/OneDrive/Desktop/Dogs-Cats/Prove/dog.jpg')
plt.imshow(img)
plt.axis('off')
plt.show()

```

```
# Aggiungi batch dimensione [1, 200, 200, 3]
img_batch = tf.expand_dims(img, 0)

# Previsione
prediction = final_model.predict(img_batch)

# Interpreta risultato
if prediction[0][0] > 0.5:
    print(f"L'immagine è classificata come: Cane ({prediction[0][0]:.2f})")
else:
    print(f"L'immagine è classificata come: Gatto ({1 - prediction[0][0]:.2f})")
```



1/1 ————— 0s 284ms/step
L'immagine è classificata come **cat** (0.86)



1/1 ————— 0s 78ms/step
L'immagine è classificata come: Cane (1.00)

In []:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js